

Informe — Laboratorio 3: MQTT hacia Azure IoT Central

Curso: IoT + Cloud + Sistemas Distribuidos — UAB

Autores: Juan Felipe Caro, Santiago Cabeza Mendez

Fecha: 24 de septiembre de 2026

App de IoT Central: UNAB-Ambiental — Monitoreo de Espacios Académicos
(laboratorio1caro.azureiotcentral.com)

Cómputo: VM nueva y aislada laboratorio3caro (Azure for Students, rg-laboratorio3caro , Mexico Central), creada de 0 para este laboratorio — sin tocar la máquina compartida del profesor ni VMs de otros compañeros.

Repositorio: github.com/felipecaronino12-cmyk/laboratorio3-mqtt-iot-central

Proyecto Wokwi: wokwi.com/projects/474815106090292225

1. Diagrama del flujo MQTT

```
Dispositivo (SDK / MQTT explícito / Wokwi)
|
| 1. DPS: HTTPS + SAS -> global.azure-devices-provisioning.net
|    (idScope + registrationId, token firmado HMAC-SHA256)
|
v
DPS responde: IoT Hub asignado
(iotc-631aa800-...-...azure-devices.net - mismo hub para toda la app)
|
| 2. MQTT 3.1.1 sobre TLS 1.2, puerto 8883
|    CONNECT user={hub}/{deviceId}?api-version=2021-04-12
|    pass=SAS(sr={hub}/devices/{deviceId})
|
v
IoT Hub — PUBLISH devices/{deviceId}/messages/events/ -> IoT Central
          (telemetry: humidity, illuminance, temperature)
^
|
| SUBSCRIBE $iothub/twin/res/# y devices/{deviceId}/messages/devicebound/#
|    (respuesta de twin y mensajes C2D / comandos)
```

Confirmado con las 3 vías (SDK, MQTT explícito, Wokwi): el "dispositivo" real de IoT Central nunca habla directo con IoT Central — siempre pasa primero por DPS (que solo resuelve a qué IoT Hub pertenece) y luego mantiene una sesión MQTT/TLS persistente contra ese hub. IoT Central es la capa de aplicación sobre ese mismo IoT Hub.

2. Los tres caminos

Camino	Dónde corre	Qué se ve
SDK <code>azure-iot-device</code> (<code>sdk_baseline.py</code>)	VM <code>laboratorio3caro</code> , dispositivo <code>python-vm-01</code> (mismo del Lab 2)	El SDK esconde topics y token; solo se ve <code>send_message()</code> .
MQTT explícito (<code>mqtt_explicito.py</code> , paho- mqtt)	VM <code>laboratorio3caro</code> , dispositivo nuevo <code>mqtt- explicito-vm</code>	SAS generado a mano (<code>dps_sas.py</code>), topic y QoS explícitos.
Arduino/ESP32 (Wokwi)	Wokwi, dispositivo <code>esp32-wokwi- 02</code> (mismo del Lab 2, firmware sin cambios)	MQTT/TLS a mano en C++ (mismo patrón que <code>mqtt_explicito.py</code>).

Los tres corren sobre la misma app y la misma plantilla del Laboratorio 1/2; `mqtt-explicito-vm` es un dispositivo nuevo (mismo `id_scope` , misma plantilla) para poder distinguir sus mensajes en el Data explorer sin mezclarlos con los del SDK.

3. Mediciones (Etapa 3)

CSV crudos en la carpeta `mediciones/` del repositorio.

3.1 Payload, intervalo y latencia (condiciones normales)

Camino	Payload (bytes)	Intervalo configurado	Intervalo observado (prom.)	Latencia local (prom.)
SDK (<code>python- vm-01</code>)	61	10 s	~10.16 s	~0.14 s (<code>send_message</code>)
MQTT explícito QoS 1	61	10 s	~10.14 s	~0.13 s (<code>wait_for_publish</code>)
MQTT explícito QoS 0	61	10 s	10 s	~0.0003 s (no espera ack)

El intervalo observado es ~1.5 % mayor que el configurado en ambos casos — overhead normal de armar el payload y la llamada de red, no un problema del protocolo. El payload de 61 bytes es fijo porque los 3 valores del modelo (`humidity` , `illuminance` , `temperature`) siempre se serializan con 1 decimal.

3.2 Tiempo hasta verse en IoT Central

En todas las pruebas el mensaje apareció en **Datos sin procesar** dentro del mismo segundo del publish local (diferencia ≤ 1 s, límite de resolución de la marca de tiempo que muestra el portal). No hay retraso perceptible entre el `PUBACK` del hub y la visibilidad en la app.

3.3 Corte de red de ~20 s

Se cortó el tráfico saliente al puerto 8883 con `iptables -A OUTPUT -p tcp --dport 8883 -j DROP` durante 20 s mientras cada cliente publicaba, y se restauró. Logs completos en el repositorio (`mediciones/sdk_cutoff.log` , `mediciones/mqtt_cutoff.log`).

Camino	Qué pasó durante el corte	Recuperación
SDK	El <code>send_message()</code> que coincidió con el corte tardó 16.09 s en vez de ~0.14 s (bloqueado esperando el ack); no hubo excepción ni reconexión visible en el log (una sola línea "Conectado").	Automática: siguiente publish salió ~16 s más tarde de lo normal y luego el intervalo volvió a 10 s.
MQTT explícito	El publish que coincidió con el corte agotó el timeout de <code>wait_for_publish(10 s)</code> sin confirmar; el script igual siguió su loop.	Automática: <code>loop_start()</code> de paho-mqtt corre <code>loop_forever()</code> internamente, que reintenta a nivel de socket. El siguiente publish, ~20 s después (duración exacta del corte), volvió a completarse en ~0.14 s normales.

Hallazgo: un `DROP` de firewall (paquetes descartados en silencio, sin `RST`) no genera un ciclo completo de "MQTT disconnect → reconnect" en ninguno de los dos clientes — ambos lo toleran a nivel de socket/TCP, con un pico de latencia proporcional a la duración del corte, sin perder mensajes ni requerir intervención manual. Esto es distinto de una desconexión real de red (p. ej. WiFi caído), donde sí se esperaría ver un `CONNACK` nuevo — ese es justamente el ciclo Conectado ↔ Desconectado que se observa en Wokwi cuando el gateway IoT de Wokwi se reinicia.

3.4 QoS 0, 1 y 2

QoS pedido	Resultado	Latencia local
0	Aceptado. "Fire and forget": el cliente no espera nada del broker.	~0.0003 s
1	Aceptado. Espera <code>PUBACK</code> .	~0.13 s
2	Aceptado sin error , pero con latencia prácticamente idéntica a QoS 1 (~0.13–0.14 s), no el handshake completo de 4 pasos	~0.13 s

Conclusión de QoS: el comportamiento observado confirma lo documentado por Microsoft: **IoT Hub/Central soporta QoS 0 y QoS 1, pero no implementa QoS 2** — en vez de rechazar la conexión o el publish, lo degrada silenciosamente a un comportamiento equivalente a QoS 1 (una sola confirmación, sin el handshake extendido). Para este escenario (telemetría ambiental cada 10 s, sin transacciones críticas) **QoS 1 es la opción correcta:** garantiza al menos una entrega sin el costo de latencia de un handshake que, de todas formas, el hub no ofrece.

4. Comparación y conclusión

Criterio	SDK <code>azure-iot-device</code>	MQTT explícito (<code>paho-mqtt</code>)	Wokwi Arduino
Facilidad de implementación	Alta: <code>create_from_symmetric_key</code> + <code>connect()</code> + <code>send_message()</code> .	Media: SAS, DPS y topics a mano en Python, pero directo.	Baja: todo lo anterior en C++, con HMAC/base64 manuales vía <code>mbedtls</code> , sin SDK oficial.
Control del protocolo	Ninguno — todo escondido.	Total — QoS por mensaje, token visible, topic exacto.	Total, igual que MQTT explícito en Python.
Payload	61 bytes	61 bytes	61 bytes
Latencia percibida	~0.14 s normal; picos de hasta 16 s ante corte de 20 s.	~0.13 s en QoS 1; ~0.0003 s en QoS 0.	Comparable (mismo transporte); no medida en detalle por la lentitud del emulador.
Reconexión ante corte de ~20 s	Automática, sin código adicional.	Automática vía <code>loop_start()</code> / <code>loop_forever()</code> de <code>paho-mqtt</code> .	En Lab 2 se observó un ciclo real Conectado ↔ Desconectado (reinicio del

			gateway de Wokwi).
Utilidad para el Lab 1 (flota de aulas)	La mejor opción en VM/servidor: menos código, menos superficie de error.	Útil para depuración/diagnóstico o QoS o de alta frecuencia.	Obligatorio para el nodo embebido real: no hay SDK oficial de Azure para Arduino.

Conclusión accionable: para un nodo que corre en una VM o servidor con un lenguaje soportado por el SDK oficial, usar siempre el SDK — resuelve DPS, renovación de tokens y reconexión sin código adicional, y el "costo" de no controlar el protocolo no aplica en un escenario de telemetría estándar como el del Laboratorio 1. Reservar un cliente MQTT explícito para: (a) plataformas sin SDK oficial (firmware embebido tipo el ESP32 de Wokwi, donde es la única opción), o (b) depuración puntual cuando hay que confirmar exactamente qué token, topic o código de error está devolviendo el broker.

5. Evidencias

Carpeta `evidencias/` del repositorio: transcripciones de "Datos sin procesar" del Data explorer para `python-vm-01`, `mqtt-explicito-vm` y `esp32-wokwi-02`, incluyendo el ciclo Conectado ↔ Desconectado de Wokwi. Logs crudos de conexión/publish en `mediciones/`.